

Gestor Financiero Personal

Tree:

Documentation Structure:

- └─ App.java
- └─ Categoria.java
- └─ categorias.java
- └─ Gastos.java
- └─ graficos.java
- └─ Ingresos.java
- └─ presupuesto.java
- └─ SQLiteDatabaseExample.java
- └─ SystemInfo.java
- └─ verIngresos.java
- └─ verPresupuestos.java

Content:

File: App.java

```
package com.mycompany.gestorfinanciero;
```

```
import javafx.application.Application;
```

```
import javafx.scene.Scene;
```

```
import javafx.scene.control.Label;
```

```
import javafx.scene.layout.StackPane;
```

```
import javafx.stage.Stage;
```

```
/**
```

- JavaFX App

```
*/
```

```
public class App extends Application {
```

```
    @Override
```

```
    public void start(Stage stage) {
```

```
        var javaVersion = SystemInfo.javaVersion();
```

```
        var javafxVersion = SystemInfo.javafxVersion();
```

```
        var label = new Label("Hello, JavaFX " + javafxVersion + ", running on Java " + javaVersion + ".");
```

```
        var scene = new Scene(new StackPane(label), 640, 480);
```

```
        stage.setScene(scene);
```

```
        stage.show();
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        launch();
```

```
    }
```

```
}
```

=====

File: Categoria.java

```
package com.mycompany.gestorfinanciero;
```

```
public class Categoria {
```

```
    private String nombre;
```

```
private boolean balance;
```

```
public Categoria(String nombre, boolean balance) {
    this.nombre = nombre;
    this.balance = balance;
}

public String getNombre() {
    return nombre;
}

public boolean isBalance() {
    return balance;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public void setBalance(boolean balance) {
    this.balance = balance;
}

@Override
public String toString() {
    return "Categoria{" + "nombre=" + nombre + ", balance=" + balance + '}';
}
```

```
}
```

=====

File: categorias.java

```
package com.mycompany.gestorfinanciero;
```

```
import java.sql.Connection;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import javafx.application.Application;
```

```
import javafx.event.ActionEvent;
```

```
import javafx.geometry.Insets;
```

```
import javafx.scene.Node;
```

```
import javafx.scene.Scene;
```

```
import javafx.scene.control.Alert;
```

```
import javafx.scene.control.Alert.AlertType;
```

```
import javafx.scene.control.Button;
```

```
import javafx.scene.control.Label;
```

```
import javafx.scene.control.TextField;
```

```
import javafx.scene.layout.HBox;
```

```
import javafx.scene.layout.VBox;
```

```
import javafx.stage.Stage;
```

```
public class categorias extends Application {
```

```
    SQLiteDatabaseExample db = new SQLiteDatabaseExample();
```

```

public void start(Stage categorias) {

    categorias.setTitle("Categorias");

    // Informacion que se proporciona para añadir, modificar, buscar o eliminar
    Label tipoCategoria = new Label("¿Qué tipo de categoria? (ingresos/gastos)");
    TextField categoriaTipo = new TextField();
    Label categoriaNombre = new Label("Nombre de la categoria");
    TextField categoria = new TextField();

    Button anadirCategoria = new Button("Añadir");
    Button modificarCategoria = new Button("Modificar");
    Button buscarCategoria = new Button("Buscar");
    Button borrarCategoria = new Button("Borrar");

    VBox panelPadre = new VBox(80);
    panelPadre.setPadding(new Insets(10, 10, 10, 10));

    VBox panelInfo = new VBox(10);
    panelInfo.getChildren().addAll(tipoCategoria, categoriaTipo, categoriaNombre, categoria);

    HBox acciones = new HBox(40);
    acciones.getChildren().addAll(panelInfo, anadirCategoria, modificarCategoria, buscarCategoria, borrarCategoria);

    VBox ingresos = new VBox(40);
    ingresos.setPadding(new Insets(10, 10, 10, 10));

    VBox gastos = new VBox(40);
    gastos.setPadding(new Insets(10, 10, 10, 10));

    HBox tipos = new HBox(80);
    tipos.getChildren().addAll(ingresos, gastos);

    panelPadre.getChildren().addAll(acciones, tipos);

    // Declarar un ArrayList para almacenar las categorías existentes
    List<String> categoriasExistentes = new ArrayList<String>();

    // Recuperar las categorías de la base de datos y agregarlas a los VBox correspondientes
    try {
        //db.connect();
        ArrayList<Categoria> categoriasArrayList = db.listCategorias();
        for (Categoria c : categoriasArrayList) {
            Label categoriaLabel = new Label(c.getNombre());
            if (c.isBalance()) {
                ingresos.getChildren().add(categoriaLabel);
            } else {
                gastos.getChildren().add(categoriaLabel);
            }
            categoriasExistentes.add(c.getNombre());
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

        javafx.event.EventHandler<ActionEvent> agregar = new javafx.event.EventHandler<ActionEvent>() {
            public void handle(ActionEvent e) {
                String tipo = categoriaTipo.getText().toLowerCase();
                String nombre = categoria.getText();

                if (nombre.isEmpty() || (!tipo.equals("ingresos") && !tipo.equals("gastos"))))
                {
                    Alert alert = new Alert(AlertType.ERROR);
                    alert.setTitle("Error");
                    alert.setHeaderText("Debe ingresar un tipo válido (ingresos/gastos) y un nombre de categoría.");
                    alert.showAndWait();
                    return;
                }

                try {
                    boolean balance = tipo.equals("ingresos");
                    db.insertCategoria(nombre, balance);

                    Label nuevaCategoriaLabel = new Label(nombre);
                    if (balance) {
                        ingresos.getChildren().add(nuevaCategoriaLabel);
                    } else {
                        gastos.getChildren().add(nuevaCategoriaLabel);
                    }

                    Alert alert = new Alert(AlertType.INFORMATION);
                    alert.setTitle("Éxito");
                    alert.setHeaderText("Categoría añadida exitosamente.");
                    alert.showAndWait();
                } catch (Exception ex) {
                    ex.printStackTrace();
                    Alert alert = new Alert(AlertType.ERROR);
                    alert.setTitle("Error");
                    alert.setHeaderText("No se pudo añadir la categoría.");
                    alert.showAndWait();
                }
            }
        };

        javafx.event.EventHandler<ActionEvent> modificar = new javafx.event.EventHandler<ActionEvent>() {
            public void handle(ActionEvent e) {
                // Implementar la lógica de modificación aquí
                categoria.getText();
                if (ingresos.getChildren().contains(new Label(categoria.getText()))) {
                    ingresos.getChildren().remove(new Label(categoria.getText()));
                    categoriasExistentes.remove(categoria.getText());
                    try {
                        //db.updateCategoriaName((categoria.getText(), true);
                        ingresos.getChildren().add(new Label(categoria.getText()));
                        categoriasExistentes.add(categoria.getText());
                    } catch (Exception ex) {
                        ex.printStackTrace();
                        Alert alert = new Alert(AlertType.ERROR);
                        alert.setTitle("Error");

```

```

        alert.setHeaderText("No se pudo modificar la categoría.");
        alert.showAndWait();
    }
} else if (gastos.getChildren().contains(new Label(categoría.getText()))) {
    gastos.getChildren().remove(new Label(categoría.getText()));
    categoríasExistentes.remove(categoría.getText());
    try {
        //db.updateCategoríaName((categoría.getText(), true);
        ingresos.getChildren().add(new Label(categoría.getText()));
        categoríasExistentes.add(categoría.getText());
    } catch (Exception ex) {
        ex.printStackTrace();
        Alert alert = new Alert(AlertType.ERROR);
        alert.setTitle("Error");
        alert.setHeaderText("No se pudo modificar la categoría.");
        alert.showAndWait();
    }
} else {
    // Muestra un mensaje de error si la categoría no existe
    Alert alert = new Alert(AlertType.ERROR);
    alert.setTitle("Error");
    alert.setHeaderText("La categoría no existe o no se ha seleccionado una categoría.");
    alert.showAndWait();
}
}
};

javafx.event.EventHandler<ActionEvent> buscar = new javafx.event.EventHandler<ActionEvent>() {
    public void handle(ActionEvent e) {
        // Implementar la lógica de búsqueda aquí
        try {
            String c = db.getCategoríaByNameString(categoría.getText());
            if (c != null) {
                Alert alert = new Alert(AlertType.INFORMATION);
                alert.setTitle("Información");
                alert.setHeaderText(c.toString());
                alert.showAndWait();
            } else {
                Alert alert = new Alert(AlertType.ERROR);
                alert.setTitle("Error");
                alert.setHeaderText("La categoría no existe.");
                alert.showAndWait();
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            Alert alert = new Alert(AlertType.ERROR);
            alert.setTitle("Error");
            alert.setHeaderText("No se pudo buscar la categoría.");
            alert.showAndWait();
        }
    }
};

javafx.event.EventHandler<ActionEvent> borrar = new javafx.event.EventHandler<ActionEvent>() {
    public void handle(ActionEvent e) {

```

```

        if (ingresos.getChildren().contains(new Label(categoria.getText()))) {
            ingresos.getChildren().remove(new Label(categoria.getText()));
            categoriasExistentes.remove(categoria.getText());
            try {
                db.deleteCategoriaByName(categoria.getText());
            } catch (Exception ex) {
                ex.printStackTrace();
                Alert alert = new Alert(AlertType.ERROR);
                alert.setTitle("Error");
                alert.setHeaderText("No se pudo borrar la categoría.");
                alert.showAndWait();
            }
        } else if (gastos.getChildren().contains(new Label(categoria.getText()))) {
            gastos.getChildren().remove(new Label(categoria.getText()));
            categoriasExistentes.remove(categoria.getText());
            try {
                db.deleteCategoriaByName(categoria.getText());
            } catch (Exception ex) {
                ex.printStackTrace();
                Alert alert = new Alert(AlertType.ERROR);
                alert.setTitle("Error");
                alert.setHeaderText("No se pudo borrar la categoría.");
                alert.showAndWait();
            }
        } else {
            // Muestra un mensaje de error si la categoría no existe
            Alert alert = new Alert(AlertType.ERROR);
            alert.setTitle("Error");
            alert.setHeaderText("La categoría no existe o no se ha seleccionado una categoría.");
            alert.showAndWait();
        }
    }
};

anadirCategoria.setOnAction(agregar);
modificarCategoria.setOnAction(modificar);
buscarCategoria.setOnAction(buscar);
borrarCategoria.setOnAction(borrar);

Scene scene = new Scene(panelPadre, 800, 600);
categorias.setScene(scene);
categorias.show();
}

public static void main(String[] args) {
    launch(args);
}

```

```

}

```

=====

File: Gastos.java

```

/*

```

- Click <nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt> to change this license

- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```

*/
package com.mycompany.gestorfinanciero;

/**
 *
 *
 * @author JIANG XIAO QI
 */

public class Gastos {
    private String nombre, categoria;
    private double importe;

    public Gastos(String nombre, String categoria, double importe) {
        this.nombre = nombre;
        this.categoria = categoria;
        this.importe = importe;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public void setImporte(double importe) {
        this.importe = importe;
    }

    @Override
    public String toString() {
        return "Gastos{" + "nombre=" + nombre + ", categoria=" + categoria + ", importe=" + importe + '}';
    }
}

```

=====

File: graficos.java

```

/*
 *
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.gestorfinanciero;

/**
 *
 *
 * @author JIANG XIAO QI
 */

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.chart.PieChart;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;

```

```
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class graficos extends Application {
```

```
    public static void main(String[] args) {
        launch(args);
    }
    private TextField presupuestoField, inversionesField, salarioField, comidaField,
```

```
entretenimientoField, higieneYsaludField, viajesField, finanzasField, otrosField;
```

```
@Override
public void start(Stage primaryStage) {
    primaryStage.setTitle("Gestión de Presupuesto");

    Button calculateButton = new Button("Calcular");

    PieChart pieChart = new PieChart();

    calculateButton.setOnAction(e -> {
        double budget = Double.parseDouble(presupuestoField.getText());
        double inversiones = Double.parseDouble(inversionesField.getText());
        double salario = Double.parseDouble(salarioField.getText());
        double comida = Double.parseDouble(comidaField.getText());
        double entretenimiento = Double.parseDouble(entretenimientoField.getText());
        double higieneYsalud = Double.parseDouble(higieneYsaludField.getText());
        double viajes = Double.parseDouble(viajesField.getText());
        double finanzas = Double.parseDouble(finanzasField.getText());
        double otros = Double.parseDouble(otrosField.getText());

        double income = inversiones + salario;
        double expenses = comida + entretenimiento + higieneYsalud + viajes + finanzas + o
tros;

        PieChart.Data expenseData = new PieChart.Data("Gastos", expenses);
        PieChart.Data incomeData = new PieChart.Data("Ingresos", income);

        pieChart.getData().setAll(expenseData, incomeData);
    });

    VBox layout = new VBox(10);
    layout.getChildren().addAll(calculateButton, pieChart);

    Scene scene = new Scene(layout, 400, 400);
    primaryStage.setScene(scene);
    primaryStage.show();
}
}
```

```
}
```

```
=====
```

File: Ingresos.java

```
/*
```

- Click <nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt> to change this license

- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```
*/
package com.mycompany.gestorfinanciero;
```

```
/**
```

```
*
```

- @author JIANG XIAO QI

```
*/
```

```
public class Ingresos {
    private String nombre, categoria;
    private double importe;
```

```
    public Ingresos(String nombre, String categoria, double importe) {
        this.nombre = nombre;
        this.categoria = categoria;
        this.importe = importe;
    }

    public String getNombre() {
        return nombre;
    }

    public String getCategoria() {
        return categoria;
    }

    public double getImporte() {
        return importe;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public void setImporte(double importe) {
        this.importe = importe;
    }

    @Override
    public String toString() {
        return "Ingresos{" + "nombre=" + nombre + ", categoria=" + categoria + ", importe=" +
            importe + '}';
    }
}
```

```
}
```

=====

File: presupuesto.java

```
/*
```

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license

- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```
*/
package com.mycompany.gestorfinanciero;

/**
 *
 *
 * @author JIANG XIAO QI
 */
```

```
//import java.beans.EventHandler;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.format.DateTimeFormatter;

import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class presupuesto extends Application {
```

```
    public TextField
```

```
    presupuestoField,salarioField,inversionesField,comidaField,entretenimientoField,higieneYsaludField,viajesField,finanzasl
    ield,otrosField;
    public CheckBox salario,inversiones,comida,entretenimiento,higieneYsalud,viajes,finanzas,otros;
```

```
    private String directory = "./Gestion de finanzas";

    public void start(Stage primaryStage) {
        primaryStage.setTitle("Nuevo presupuesto");

        // Crear componentes de la interfaz de usuario
        Label presupuestoLabel = new Label("Presupuesto:");
        presupuestoField = new TextField();

        salario = new CheckBox("Salario");
        salarioField = new TextField();
        inversiones = new CheckBox("Inversiones");
        inversionesField = new TextField();
        comida = new CheckBox("Comida");
        comidaField = new TextField();
        entretenimiento = new CheckBox("Entretenimiento");
        entretenimientoField = new TextField();
        higieneYsalud = new CheckBox("Higiene y salud");
        higieneYsaludField = new TextField();
        viajes = new CheckBox("Viajes");
        viajesField = new TextField();
        finanzas = new CheckBox("Finanzas");
        finanzasField = new TextField();
```

```

otros = new CheckBox("Otros");
otrosField = new TextField();

Button aceptarButton = new Button("Aceptar");

// Crear un manejador de eventos para el botón aceptar
javafx.event.EventHandler<ActionEvent> aceptar = new javafx.event.EventHandler<ActionEvent>() {
    public void handle(ActionEvent e) {
        SQLiteDatabaseExample dbExample = new SQLiteDatabaseExample();

        try {
            dbExample.connect();

            if (salario.isSelected()) {
                double cantidad = Double.parseDouble(salarioField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Salario");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (inversiones.isSelected()) {
                double cantidad = Double.parseDouble(inversionesField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Inversiones");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (comida.isSelected()) {
                double cantidad = Double.parseDouble(comidaField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Comida");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (entretenimiento.isSelected()) {
                double cantidad = Double.parseDouble(entretenimientoField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Entretenimiento");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (higieneYsalud.isSelected()) {
                double cantidad = Double.parseDouble(higieneYsaludField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Higiene y Salud");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (viajes.isSelected()) {
                double cantidad = Double.parseDouble(viajesField.getText());
                int categoriaId = dbExample.obtenerCategoriaId("Viajes");
                if (categoriaId != -1) {
                    dbExample.insertarPresupuesto(categoriaId, cantidad);
                }
            }
            if (finanzas.isSelected()) {
                double cantidad = Double.parseDouble(finanzasField.getText());
            }
        }
    }
};

```

```

        int categoriaId = dbExample.obtenerCategoriaId("Finanzas");
        if (categoriaId != -1) {
            dbExample.insertarPresupuesto(categoriaId, cantidad);
        }
    }
    if (otros.isSelected()) {
        double cantidad = Double.parseDouble(otrosField.getText());
        int categoriaId = dbExample.obtenerCategoriaId("Otros");
        if (categoriaId != -1) {
            dbExample.insertarPresupuesto(categoriaId, cantidad);
        }
    }
} catch (Exception ex) {
    ex.printStackTrace();
}
}
};

// Asociamos la accion al boton
aceptarButton.setOnAction(aceptar);

// Crear un diseño VBox para organizar los componentes
VBox layout = new VBox(10);
layout.setPadding(new Insets(10, 10, 10, 10));
layout.getChildren().addAll(presupuestoLabel,

```

presupuestoField,salario,salarioField,inversiones,inversionesField,comida,comidaField,entretenimiento,entretenimientoField,higieneYsalud,higieneYsaludField,viajes,viajesField,finanzas,finanzasField,otros,otrosField,aceptarButton);

```

// Crear la escena y mostrarla
Scene scene = new Scene(layout, 600, 600);
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}

```

}

=====

File: SQLiteDatabaseExample.java

/*

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

*/

package com.mycompany.gestorfinanciero;

/**

*

- @author JIANG XIAO QI

*/

import java.sql.*;

import java.util.ArrayList;

```

public class SQLiteDatabaseExample {

    public static void main(String[] args) {
        SQLiteDatabaseExample example = new SQLiteDatabaseExample();
        example.createTables();
        //example.insertDefaultCategories();

        /* Ejemplos de operaciones CRUD para categorias -> TO DELETE
        example.insertCategoria("NuevaCategoria", false);
        example.updateCategoria(1, "CategoriaActualizada", true);
        example.listCategorias();
        example.getCategoriaById(1);
        example.deleteCategoria(1);*/

        /* Ejemplos de operaciones CRUD para gastos -> TO DELETE
        example.insertGasto(1, "NuevoGasto", 50.0, 30.0);
        example.updateGasto(1, "GastoActualizado", 60.0, 40.0);
        example.listGastos();
        example.getGastoById(1);
        example.deleteGasto(1);*/
    }

    public Connection connect() {
        String url = "jdbc:sqlite:mi_base_de_datos.db";
        Connection conn = null;
        try {
            conn = DriverManager.getConnection(url);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return conn;
    }

    public void createTables() {
        String createCategoriasTableSQL = "CREATE TABLE IF NOT EXISTS categorias ("
            + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
            + "nombre VARCHAR(20) NOT NULL,"
            + "balance BOOLEAN NOT NULL)";

        String createGastosTableSQL = "CREATE TABLE IF NOT EXISTS gastos ("
            + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
            + "categoria_id INTEGER NOT NULL,"
            + "nombre VARCHAR(20) NOT NULL,"
            + "importe DOUBLE NOT NULL,"
            + "capital DOUBLE NOT NULL,"
            + "FOREIGN KEY (categoria_id) REFERENCES categorias(id))";

        String createPresupuestosTableSQL = "CREATE TABLE IF NOT EXISTS presupuestos ("
            + "categoria_id INTEGER NOT NULL,"
            + "cantidad DOUBLE NOT NULL,"
            + "FOREIGN KEY (categoria_id) REFERENCES categorias(id))";

        try (Connection conn = connect();
            Statement stmt = conn.createStatement()) {
            stmt.execute(createCategoriasTableSQL);
            stmt.execute(createGastosTableSQL);
            stmt.execute(createPresupuestosTableSQL);
        } catch (SQLException e) {
    
```

```

        e.printStackTrace();
    }
}

public void insertDefaultCategories() {
    insertCategoria("salario", true);
    insertCategoria("inversiones", false);
    insertCategoria("comida", false);
    insertCategoria("entretenimiento", false);
    insertCategoria("salud", false);
    insertCategoria("viajes", false);
    insertCategoria("finanzas", false);
    insertCategoria("otros", false);
}

public int obtenerCategoriaId(String nombreCategoria) throws SQLException {
    String sql = "SELECT id FROM categorias WHERE nombre = ?";

    try (Connection conn = this.connect();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, nombreCategoria);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            return rs.getInt("id");
        }
    } catch (SQLException e) {
        System.out.println(e.getMessage());
    }

    return -1; // Retorna -1 si no se encuentra la categoría
}

public void insertarPresupuesto(int categoriaId, double cantidad) throws SQLException {
    String sql = "INSERT INTO presupuestos (categoria_id, cantidad) VALUES (?, ?)";

    try (Connection conn = this.connect();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, categoriaId);
        pstmt.setDouble(2, cantidad);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.out.println(e.getMessage());
    }
}

// Operaciones CRUD para Categorías

public void insertCategoria(String nombre, boolean balance) {
    String insertCategoriaSQL = "INSERT INTO categorias (nombre, balance) VALUES (?, ?)";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(insertCategoriaSQL)) {
        pstmt.setString(1, nombre);
        pstmt.setBoolean(2, balance);
        pstmt.executeUpdate();
    } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
}

public void updateCategoria(int id, String nombre, boolean balance) {
    String updateCategoriaSQL = "UPDATE categorias SET nombre = ?, balance = ? WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(updateCategoriaSQL)) {
        pstmt.setString(1, nombre);
        pstmt.setBoolean(2, balance);
        pstmt.setInt(3, id);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void updateCategoriaName(String nombreViejo, String nombreNuevo) {
    String updateCategoriaNameSQL = "UPDATE categorias SET nombre = ? WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(updateCategoriaNameSQL)) {
        pstmt.setString(1, nombreNuevo);
        pstmt.setString(2, nombreViejo);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public ArrayList<Categoria> listCategorias() {
    ArrayList<Categoria> categorias = new ArrayList<>();
    String selectCategoriasSQL = "SELECT * FROM categorias";

    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectCategoriasSQL)) {
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id") +
                ", Nombre: " + rs.getString("nombre") +
                ", Balance: " + rs.getBoolean("balance"));
            String nombre = rs.getString("nombre");
            boolean balance = rs.getBoolean("balance");
            categorias.add(new Categoria(nombre, balance));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return categorias;
}

public void getCategoriaById(int id) {
    String selectCategoriaByIdSQL = "SELECT * FROM categorias WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectCategoriaByIdSQL)) {
        pstmt.setInt(1, id);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {

```

```

        System.out.println("ID: " + rs.getInt("id") +
            ", Nombre: " + rs.getString("nombre") +
            ", Balance: " + rs.getBoolean("balance"));
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
}

public String getCategoriabyName(String nombre) {
    String selectCategoriaByNameSQL = "SELECT * FROM categorias WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectCategoriaByNameSQL)) {
        pstmt.setString(1, nombre);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") +
                    ", Nombre: " + rs.getString("nombre") +
                    ", Balance: " + rs.getBoolean("balance"));
                return rs.getString("nombre");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

public Categoria getCategoriaByName(String nombre) {
    String selectCategoriaByNameSQL = "SELECT * FROM categorias WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectCategoriaByNameSQL)) {
        pstmt.setString(1, nombre);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") +
                    ", Nombre: " + rs.getString("nombre") +
                    ", Balance: " + rs.getBoolean("balance"));
                Categoria categoria = new Categoria(rs.getString("nombre"), rs.getBoolean(
("balance")));
                System.out.println(categoria);
                return categoria;
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

public String getCategoriaByNameString(String nombre) {
    String selectCategoriaByNameSQL = "SELECT * FROM categorias WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectCategoriaByNameSQL)) {
        pstmt.setString(1, nombre);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {

```



```

        System.out.println("ID: " + rs.getInt("id") +
            ", Nombre: " + rs.getString("nombre") +
            ", Balance: " + rs.getBoolean("balance"));
        String categoria = rs.getString("nombre");
        System.out.println(categoria);
        return categoria;
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
return "No se encontró la categoría";
}

public void deleteCategoria(int id) {
    String deleteCategoriaSQL = "DELETE FROM categorias WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(deleteCategoriaSQL)) {
        pstmt.setInt(1, id);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void deleteCategoriaByName(String nombre) {
    String deleteCategoriaByNameSQL = "DELETE FROM categorias WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(deleteCategoriaByNameSQL)) {
        pstmt.setString(1, nombre);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// Operaciones CRUD para Gastos

public void insertGasto(int categoriaId, String nombre, double importe, double capital) {
    String insertGastoSQL = "INSERT INTO gastos (categoria_id, nombre, importe, capital) V
ALUES (?, ?, ?, ?)";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(insertGastoSQL)) {
        pstmt.setInt(1, categoriaId);
        pstmt.setString(2, nombre);
        pstmt.setDouble(3, importe);
        pstmt.setDouble(4, capital);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void updateGasto(int id, String nombre, double importe, double capital) {
    String updateGastoSQL = "UPDATE gastos SET nombre = ?, importe = ?, capital = ? WHERE
id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(updateGastoSQL)) {

```

```

        pstmt.setString(1, nombre);
        pstmt.setDouble(2, importe);
        pstmt.setDouble(3, capital);
        pstmt.setInt(4, id);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void updateGastoCapital(int id, double capital) {
    String updateGastoCapitalSQL = "UPDATE gastos SET capital = ? WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(updateGastoCapitalSQL)) {
        pstmt.setDouble(1, capital);
        pstmt.setInt(2, id);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void listGastos() {
    String selectGastosSQL = "SELECT * FROM gastos";
    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectGastosSQL)) {
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id") +
                ", Categori❖a ID: " + rs.getInt("categoria_id") +
                ", Nombre: " + rs.getString("nombre") +
                ", Importe: " + rs.getDouble("importe") +
                ", Capital: " + rs.getDouble("capital"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void getGastoById(int id) {
    String selectGastoByIdSQL = "SELECT * FROM gastos WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectGastoByIdSQL)) {
        pstmt.setInt(1, id);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") +
                    ", Categori❖a ID: " + rs.getInt("categoria_id") +
                    ", Nombre: " + rs.getString("nombre") +
                    ", Importe: " + rs.getDouble("importe") +
                    ", Capital: " + rs.getDouble("capital"));
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

public void deleteGasto(int id) {
    String deleteGastoSQL = "DELETE FROM gastos WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(deleteGastoSQL)) {
        pstmt.setInt(1, id);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

//Obtener el ultimo capital O CUANTO NOS QUEDA
public double obtenerUltimoCapital() {
    String selectUltimoCapitalSQL = "SELECT capital FROM gastos ORDER BY id DESC LIMIT 1";
    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectUltimoCapitalSQL)) {
        if (rs.next()) {
            return rs.getDouble("capital");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0.0; // Valor predeterminado si no se encuentra ninguna entrada en la tabla de
gastos
}

//Obtener gastos de una categoria
public double obtenerImporteTotalPorCategoria(int categoriaId) {
    String selectImporteTotalSQL = "SELECT SUM(importe) AS importe_total FROM gastos WHERE
categoria_id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectImporteTotalSQL)) {
        pstmt.setInt(1, categoriaId);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return rs.getDouble("importe_total");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0.0; // Valor predeterminado si no se encuentra ninguna entrada para la categor
❖a dada
}

//Listar gastos por categoria (todas)
public double obtenerImporteTotalTodasCategorias() {
    String selectImporteTotalSQL = "SELECT SUM(importe) AS importe_total FROM gastos GROUP
BY categoria_id";
    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectImporteTotalSQL)) {
        if (rs.next()) {
            return rs.getDouble("importe_total");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

    }
    return 0.0; // Valor predeterminado si no se encuentra ninguna entrada en la tabla de
gastos
}

//Listar solo gastos por categoria (balance false)
public ArrayList<Gastos> listarGastosPorCategorias() {
    ArrayList<Gastos> gastos = new ArrayList<>();
    //String selectGastosPorCategoriasSQL = "SELECT nombre, importe FROM gastos WHERE bala
nce = false";

    String selectGastosPorCategoriasSinBalanceSQL =
        "SELECT g.id, g.categoria_id, g.nombre, g.importe, g.capital, c.nombre as cate
goria_nombre " +
            "FROM gastos g " +
            "JOIN categorias c ON g.categoria_id = c.id " +
            "WHERE c.balance = false";

    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectGastosPorCategoriasSinBalanceSQL)) {

        while (rs.next()) {
            System.out.println("ID Gasto: " + rs.getInt("id") +
                ", Categorí a ID: " + rs.getInt("categoria_id") +
                ", Categorí a Nombre: " + rs.getString("categoria_nombre") +
                ", Nombre Gasto: " + rs.getString("nombre") +
                ", Importe: " + rs.getDouble("importe") +
                ", Capital: " + rs.getDouble("capital"));

            String nombre = rs.getString("nombre");
            String categoria = rs.getString("categoria");
            double importe = rs.getDouble("importe");
            gastos.add(new Gastos(nombre, categoria, importe));
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return gastos;
}

//listar ingresos por categorias (balance true)
public ArrayList<Ingresos> listarIngresosPorCategorias() {
    ArrayList<Ingresos> ingresos = new ArrayList<>();
    String selectIngresosSQL = "SELECT nombre, importe FROM gastos WHERE balance = true";

    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectIngresosSQL)) {

        while (rs.next()) {
            String nombre = rs.getString("nombre");
            String categoria = rs.getString("categoria");
            double importe = rs.getDouble("importe");
            ingresos.add(new Ingresos(nombre, categoria, importe));
        }

    }
}

```

```

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return ingresos;
}

//Obtener el balance de una categoria por id
public boolean obtenerBalancePorId(int categoriaId) {
    String selectBalanceSQL = "SELECT balance FROM categorias WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectBalanceSQL)) {
        pstmt.setInt(1, categoriaId);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return rs.getBoolean("balance");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false; // Valor predeterminado si no se encuentra ninguna entrada para la categor
    or a dada
}

//Obtener el nombre de una categoria por id
public String obtenerNombrePorId(int categoriaId) {
    String selectNombreSQL = "SELECT nombre FROM categorias WHERE id = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectNombreSQL)) {
        pstmt.setInt(1, categoriaId);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return rs.getString("nombre");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return ""; // Valor predeterminado si no se encuentra ninguna entrada para la categorí
    a dada
}

//obtener balance de una categoria por nombre
public boolean obtenerBalancePorNombre(String nombreCategoria) {
    String selectBalanceSQL = "SELECT balance FROM categorias WHERE nombre = ?";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(selectBalanceSQL)) {
        pstmt.setString(1, nombreCategoria);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return rs.getBoolean("balance");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false; // Valor predeterminado si no se encuentra ninguna entrada para la categ

```

```

    or a dada
}

//borrar todos los gastos e ingresos
public void borrarTodosLosGastosIngresos() {
    String deleteTodosLosGastosSQL = "DELETE FROM gastos";
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(deleteTodosLosGastosSQL)) {
        pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

//borrar todos los gastos (balance false)
public void borrarTodosLosGastos() {
    String selectCategoriasSinBalanceSQL = "SELECT id FROM categorias WHERE balance = false";
    String deleteGastosPorCategoriaSQL = "DELETE FROM gastos WHERE categoria_id = ?";

    try (Connection conn = connect();
        Statement stmt = conn.createStatement();
        PreparedStatement pstmt = conn.prepareStatement(deleteGastosPorCategoriaSQL)) {

        try (ResultSet rs = stmt.executeQuery(selectCategoriasSinBalanceSQL)) {
            while (rs.next()) {
                int categoriaId = rs.getInt("id");
                pstmt.setInt(1, categoriaId);
                pstmt.executeUpdate();
            }
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

```

}

```

=====

File: SystemInfo.java

```

package com.mycompany.gestorfinanciero;

```

```

public class SystemInfo {

```

```

    public static String javaVersion() {
        return System.getProperty("java.version");
    }

    public static String javafxVersion() {
        return System.getProperty("javafx.version");
    }
}

```

```

}

```

File: verIngresos.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.gestorfinanciero;

/**
 *
 * @author JIANG XIAO QI
 */
import java.util.ArrayList;
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.event.ActionEvent;
import javafx.scene.Scene;
import javafx.scene.chart.PieChart;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class verIngresos extends Application {

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Ver ingresos");

        Button calcularButton = new Button("Calcular");

        VBox gastosVB = new VBox(20); // Cambiar el espacio entre los elementos a 20
        final PieChart pieChart = new PieChart();

        javafx.event.EventHandler<ActionEvent> calcular = new javafx.event.EventHandler<Action
Event>() {
            public void handle(ActionEvent e) {
                // Inicializar la lista de ingresos
                ArrayList<Ingresos> ingresos = new ArrayList<>();

                SQLiteDatabaseExample example = new SQLiteDatabaseExample();
                try {
                    // Obtenemos las categorias de la base de datos
                    example.connect();
                    ingresos = example.listarIngresosPorCategorias();
                } catch (Exception ex) {
                    ex.printStackTrace(); // Mostrar la traza de la excepción
                    System.out.println("Error, no se ha podido acceder a la base de datos");
                }

                // Preparar los datos para el gráfico circular
                ObservableList<PieChart.Data> pieChartData = FXCollections.observableArrayList
                ();

                for (Ingresos ingreso : ingresos) {
                    String ingresoCategoria = ingreso.getCategoria();
                }
            }
        };
    }
}
```

```

        double ingresoCantidad = ingreso.getImporte();
        pieChartData.add(new PieChart.Data(ingresoCategoria, ingresoCantidad));
    }

    // Actualizar el gráfico solo si hay datos
    if (!pieChartData.isEmpty()) {
        pieChart.getData().setAll(pieChartData);
    } else {
        System.out.println("No hay datos de ingresos para mostrar.");
    }
}

};

calcularButton.setOnAction(calcular);

gastosVB.getChildren().addAll(calcularButton, pieChart);

Scene scene = new Scene(gastosVB, 800, 800);
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}

```

```

}

```

File: verPresupuestos.java

```

/*

```

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```

*/

```

```

package com.mycompany.gestorfinanciero;

```

```

/**

```

```

*

```

- @author JIANG XIAO QI

```

*/

```

```

import javafx.application.Application;
import javafx.beans.value.ObservableValue;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.Scene;
import javafx.scene.control.ListView;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;

public class verPresupuestos extends Application {

```



```

@Override
public void start(Stage primaryStage) {
    primaryStage.setTitle("Ver presupuestos");

    // Directorio que contiene los ficheros
    String directorio = "../Gestion de finanzas";

    // Lista para almacenar los nombres de archivos
    ObservableList<String> archivos = FXCollections.observableArrayList();

    // Obtén los archivos en el directorio
    File carpetaFile = new File(directorio);
    // Verifica si el directorio existe
    if (carpetaFile.exists() && carpetaFile.isDirectory()) {
        File[] files = carpetaFile.listFiles();
        // Verifica si el directorio esta vacio
        if (files != null) {
            for (File file : files) {
                if (file.isFile()) {
                    archivos.add(file.getName());
                }
            }
        }
    }

    // Crea un ListView para mostrar los archivos
    ListView<String> listView = new ListView<>();
    listView.setItems(archivos);

    // Crea un TextField de solo lectura para mostrar el contenido del archivo
    TextField fileContentField = new TextField();
    fileContentField.setEditable(false);

    // Maneja eventos de selección en el ListView
    listView.getSelectionModel().selectedItemProperty().addListener(new javafx.beans.value.ChangeListener<String>()

```

```

{
    @Override
    public void changed(ObservableValue<? extends String> observable, String oldValue, String newValue) {
        if (newValue != null) {
            String selectedFileName = newValue;
            try {
                // Carga el contenido del archivo seleccionado y muéstralo en el TextField
                String filePath = Paths.get(directorio, selectedFileName).toString();
                String fileContent = new String(Files.readAllBytes(Paths.get(filePath)));
                fileContentField.setText(fileContent);
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
});

```

```

VBox vbox = new VBox(listView);
Scene scene = new Scene(vbox, 300, 400);

```

```
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```